Segurança da Informação
Aula Rainbow Tables

Ivan Sendin

FACOM - Universidade Federal de Uberlândia
ivansendin@yahoo.com,sendin@ufu.br

13 de outubro de 2026

O que as pessoas querem dizer quando perguntam sobre "inverter um hash"?

Geralmente, quando alguém pergunta sobre "inverter um hash", eles podem estar se referindo a uma das seguintes situações:

1. **Ataques de força bruta ou dicionário:** Tentar adivinhar a entrada original (como uma senha) gerando hashes para várias entradas comuns (palavras de dicionário, combinações de caracteres) e comparando-os com o hash alvo. Se houver uma correspondência, a entrada original foi "descoberta". Isso não é inverter o hash, mas sim tentar encontrar a entrada que o gerou.
2. **Tabelas arco-íris (Rainbow Tables):** São tabelas pré-computadas que contêm milhões de pares de hash e suas correspondentes entradas originais. Elas são usadas para acelerar ataques de força bruta, especialmente contra senhas com hashes fracos ou que não usam "salt" (um valor aleatório adicionado à senha antes do hash para torná-lo único).
3. **Criptografia versus Hashing:** Confundir hashing com criptografia. A criptografia é um processo bidirecional (você pode criptografar e descriptografar), enquanto o hashing é unidirecional.

Quase acertou....

- Alguem quer inverter um hash (MD5, SHA,...)
- “Pegou” na rede...
- Hack, invasão,...
  Padrão, acontece todo dia
- Ao ”inverter”o hash a senha é obtida
- Personificação

- Dicionário gigantesco resolve o problema Coletivamente, sem duvidas!!
- Todas as combinações até um certo tamanho
- Combinações de palavras de dicionário
- Modificações
  - Upper/lower
  - Fonetica
  - $to \rightarrow 2, S \rightarrow 5$
- Caracteres especiais

- É possível baixar(comprar) um dicionário pré-computado
- Arquivo gigantesco....
- 2^{53} chaves
- São muitas chaves!
- Fim de problema...

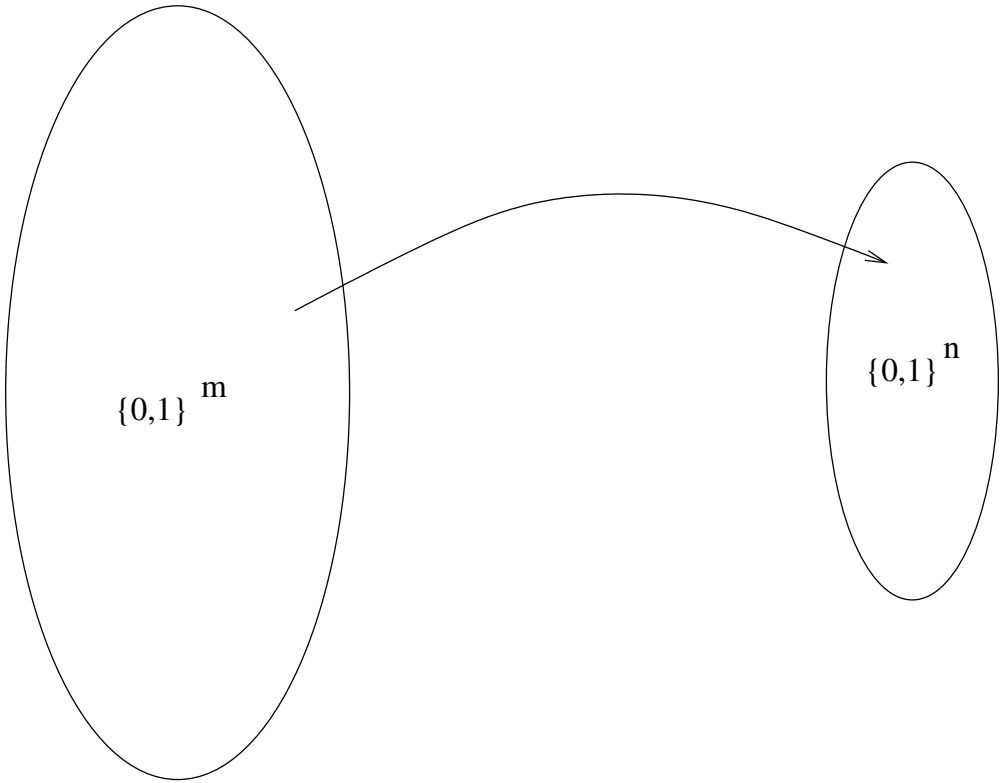
- (Neste momento alguem tem que fazer algumas contas e contestar a informação passada!)

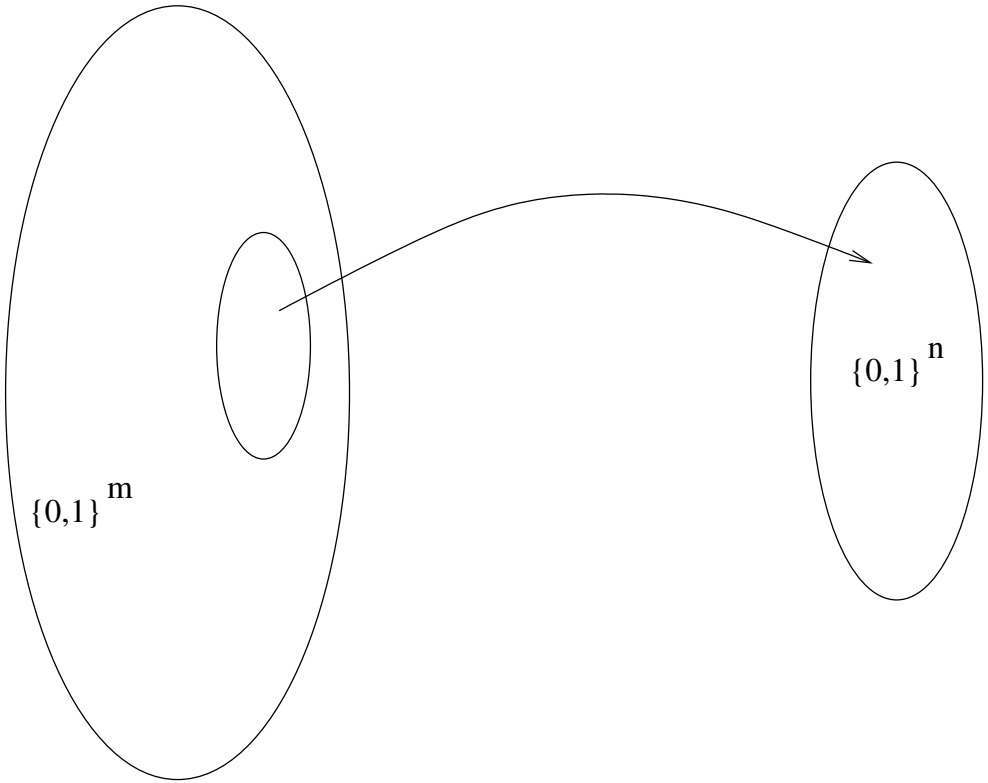
- 2^{53} chaves
- Qual é o tamanho deste arquivo??

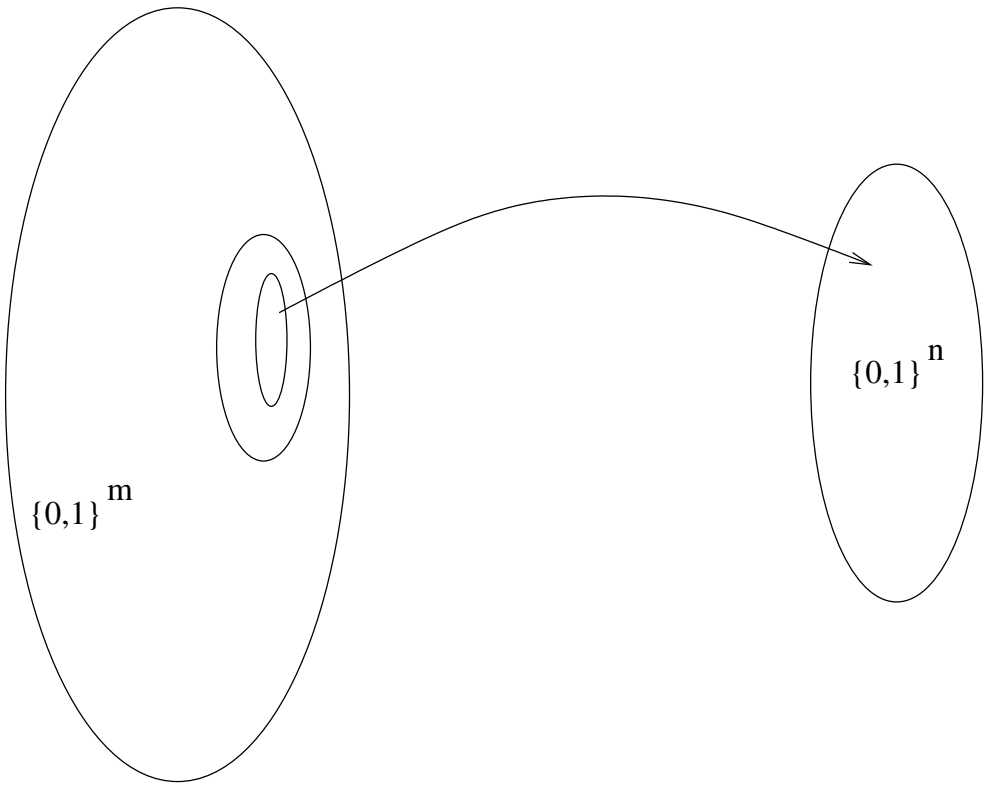
- hash de 128 bits
- (MD5 ainda é muito utilizado....)
- 128 bits é suficiente...mesmo com o Ataque do Aniversário[1]
- 128 bits = 16 bytes
- Chute: cada chave 10 bytes
- 26 bytes * 2^{53}
- $2^{4,\ldots}$ * 2^{53} bytes
- Se considerarmos algum overread temos algo muito próximo de um exabyte
  (ou um hash de 512 bits)
- (kilo, mega, giga, tera, peta, exa...)
- Compactar??

[1] Cuidado com as propriedades!

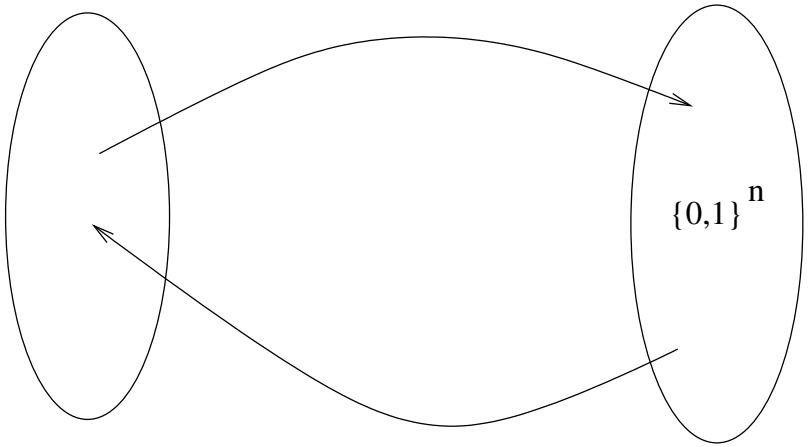
- Existe uma tecnica do tipo time-memory tradeoff
- Vamos gastar um pouco a mais de tempo....
- Para gastar **muito** menos memoria
- (exabyte p/ gigabyte)


{0,1} m
{0,1} n


{0,1} m
{0,1} n


{0,1} m
{0,1} n

- Vamos criar uma função que “volta” para o dicionario
- Alterna Dominio/Imagem
- Dicionario $\rightarrow \{0,1\}^n$
- $\{0,1\}^n \rightarrow$ Dicionario


{0,1}
n

- "Parece" com um hash
- Mas não é um hash
- **Função de REDUÇÃO**
- $\mathcal{R}$
- **Não** é a inversa do hash!
- **Tenta** uma distribuição no dicionário....
- Se o dicionario tem 100 posições

$$\mathcal{R}(h) := h\%100$$

- $\mathcal{R}$ e $\mathcal{H}$
- Alternam Dominio/Imagem
- Podemos criar uma longa cadeia....
- Dado uma s:
- $\mathcal{H} \rightarrow \mathcal{R} \rightarrow \mathcal{H} \rightarrow \mathcal{R} \rightarrow \mathcal{H} \rightarrow \mathcal{R} \ldots$

- $\mathcal{H} \rightarrow \mathcal{R} \rightarrow \mathcal{H} \rightarrow \mathcal{R} \rightarrow \mathcal{H} \rightarrow \mathcal{R}$
- Exemplo Dicionario: Alfabeto, n=5

$$A \rightarrow 00011 \rightarrow D \rightarrow 11010 \rightarrow Z \rightarrow 00110 \rightarrow E$$

- Armazeno o primeiro e o último "textos"

$$A, E$$

- (Vou chamar de "texto" as palavras do dicionario...)
- start/end

- Dado um $h = \mathcal{H}(senha)$....
- ....como saber se ele esta cadeia ?
- (Esta entre start/end)?

- Repito a operação de criação da cadeia...e comparo
- ? $11010 \in (A, E)$

$$A \rightarrow 00011 \rightarrow D \rightarrow 11010 \rightarrow Z \rightarrow 00110 \rightarrow E$$

- Descubro que $\mathcal{H}(D) = 11010$ e "inverti"o hash

- OU

- Aplico $\mathcal{R}(h)$
- Verifico se é igual ao end....
- ? $11010 \in (A, E)$

$$A \rightarrow 00011 \rightarrow D \rightarrow 11010 \rightarrow Z \rightarrow 00110 \rightarrow E$$

- Aplico $\mathcal{R}(11010) = Z$
- $Z \neq E, Z \neq A$
- Agora eu "ando"da direita para esquerda...

- ? $11010 \in (A, E)$

$$A \rightarrow 00011 \rightarrow D \rightarrow 11010 \rightarrow Z \rightarrow 00110 \rightarrow E$$

- Aplico $\mathcal{R}(11010) = Z, Z \neq E$...
- Aplico

$$\mathcal{R}(\mathcal{H}(\mathcal{R}(11010))) =$$
$$\mathcal{R}(\mathcal{H}(Z)) =$$
$$\mathcal{R}(00110) =$$
$$= E$$

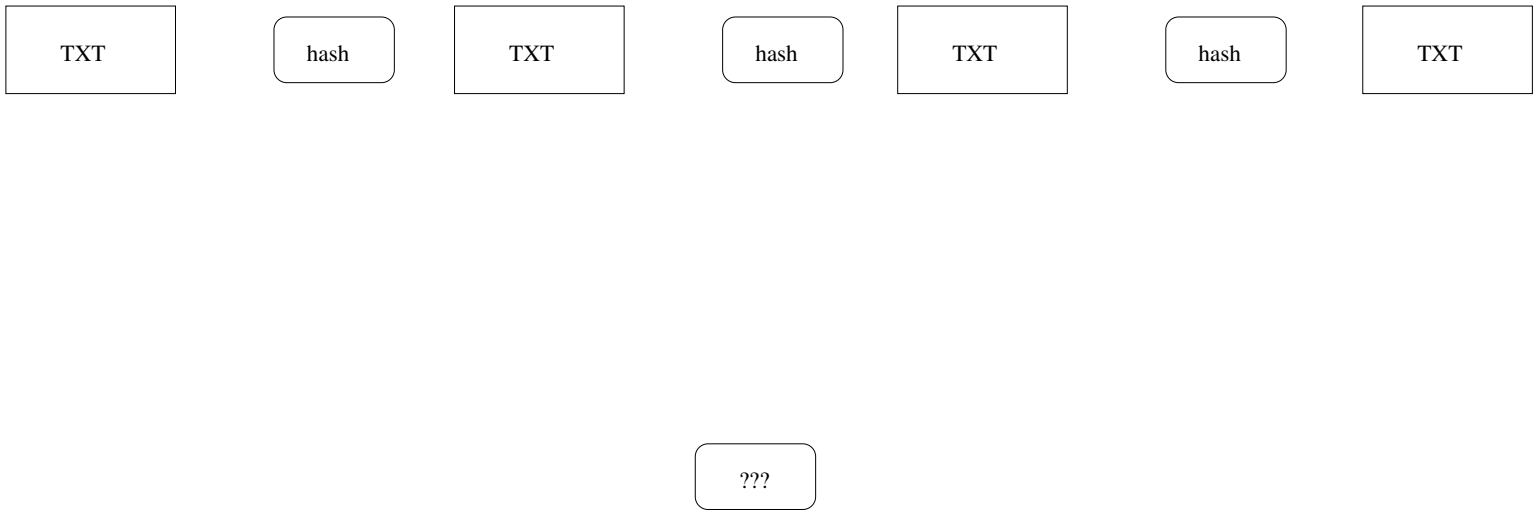
- Eu descubro que a minha busca esta entre A e E

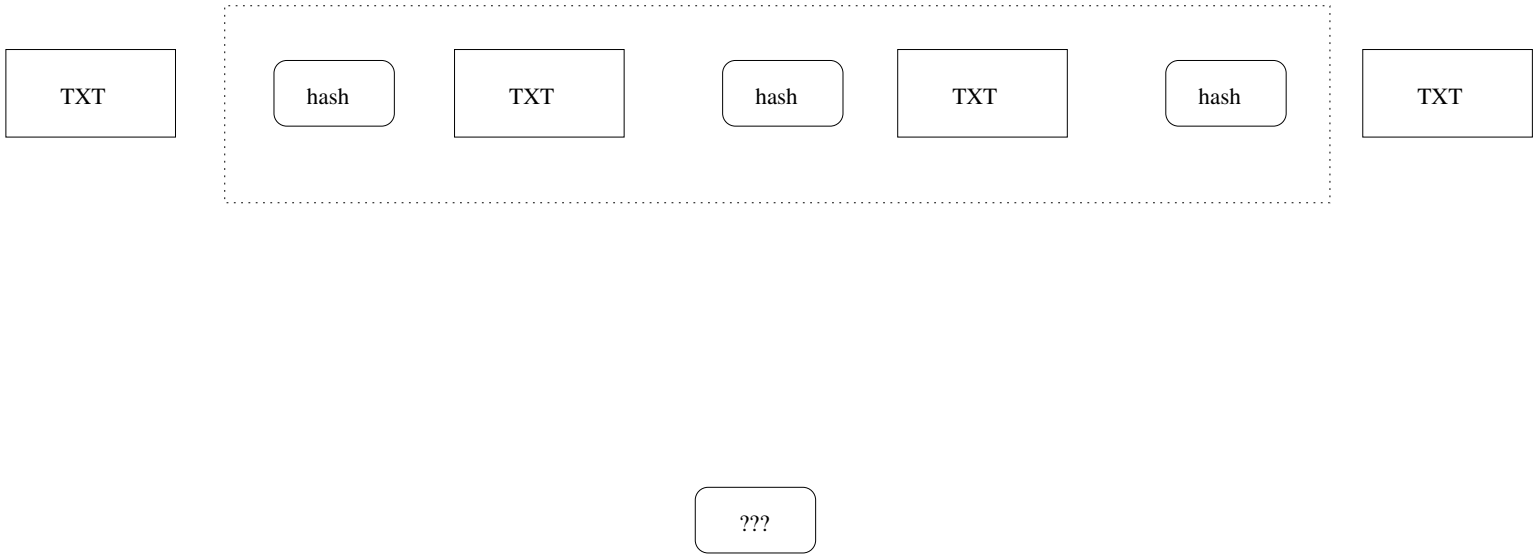
Repetindo...

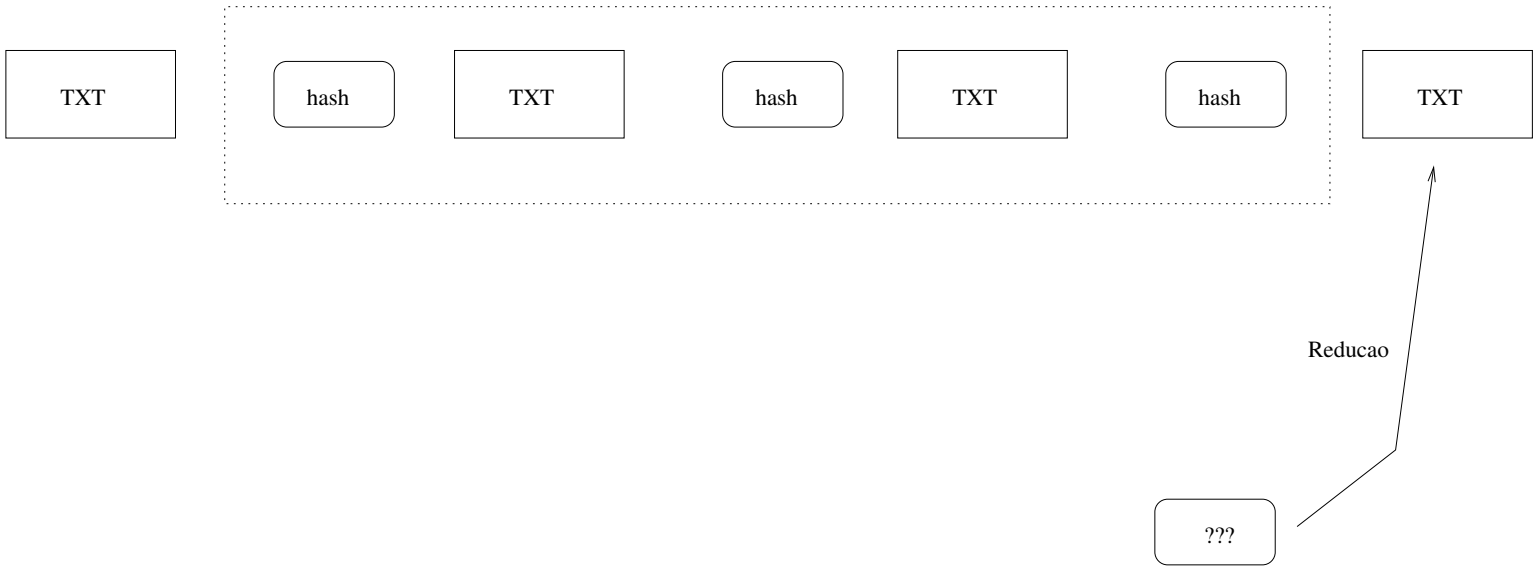
- ? $11010 \in (A, E)$

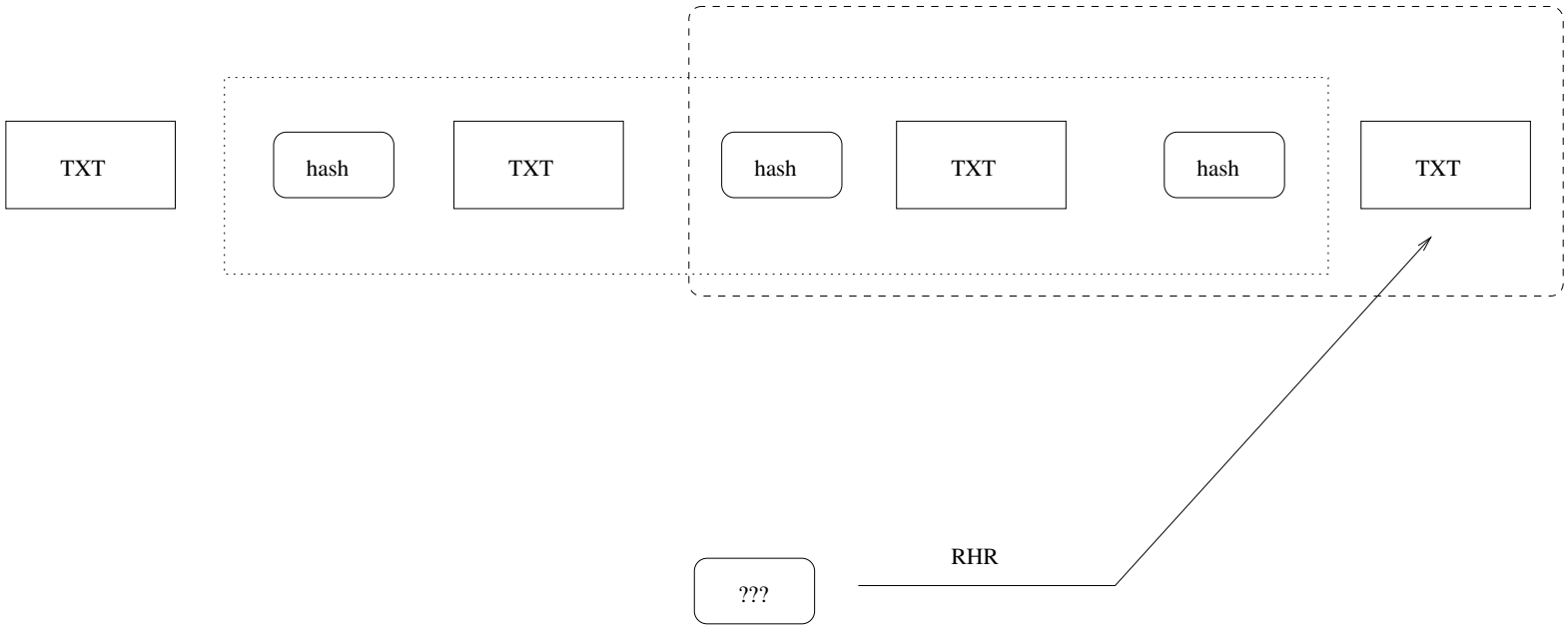
$$A \rightarrow 00011 \rightarrow D \rightarrow 11010 \rightarrow Z \rightarrow 00110 \rightarrow E$$

- $\mathcal{R}(11010)? = E$, pergunto se a minha busca esta na posição n
- $\mathcal{RHR}(11010)? = E$, pergunto se a minha busca esta na posicao n-1 (nosso exemplo..)
- $\mathcal{RHRHR}(11010)? = E$, pergunto se a minha busca esta na posicao n-2


TXT
hash
TXT
hash
TXT
hash
TXT
???

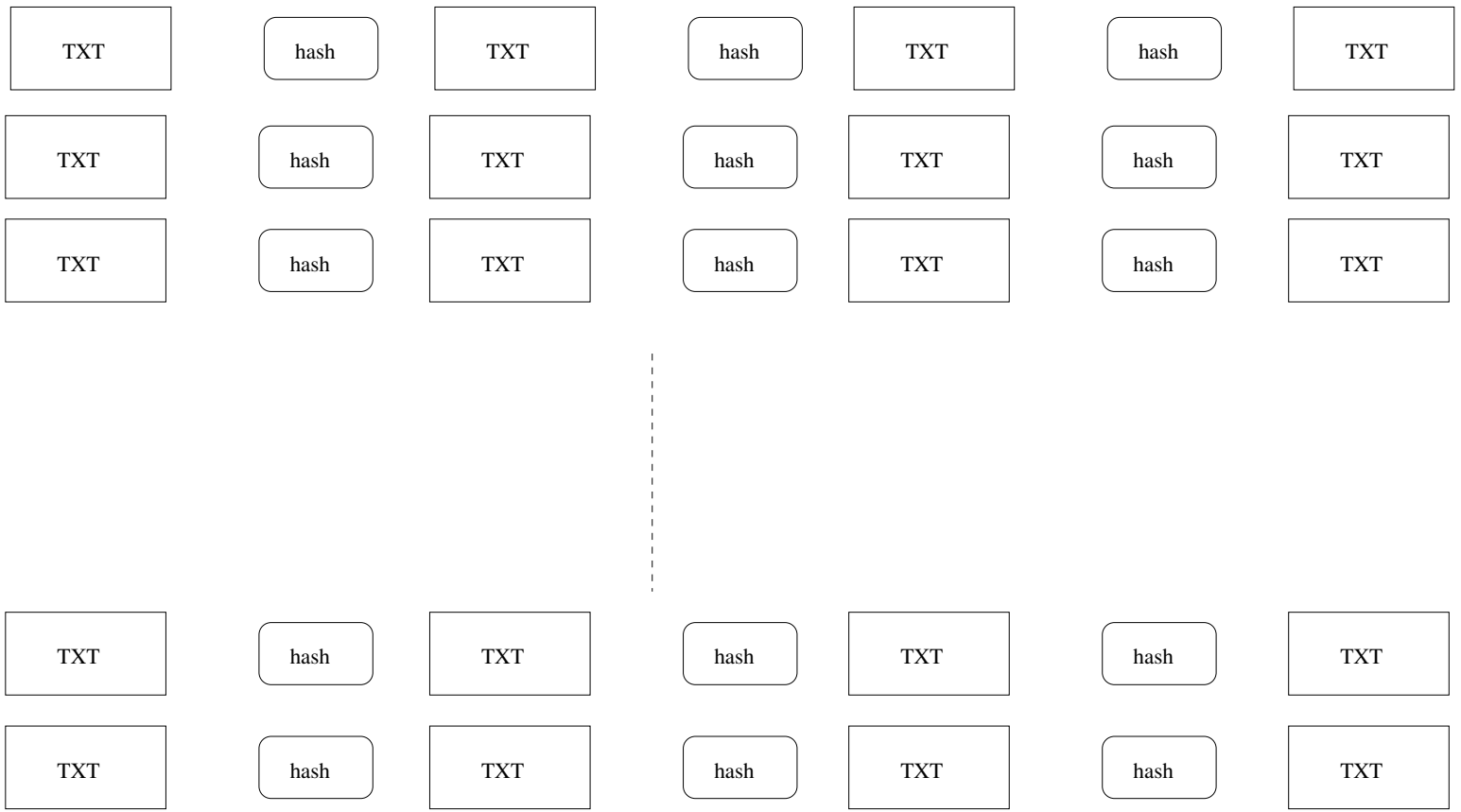

TXT
hash
TXT
hash
TXT
hash
TXT
???


TXT
hash
TXT
hash
TXT
hash
TXT
Reducao
???


TXT
hash
TXT
hash
TXT
hash
TXT
???
RHR

- **Importante:** As funções caminham pra "direita"
- $\mathcal{RHRHR}(11010) == E$
- Não é possivel chegar a D partindo de E!
- É necessário iniciar em A!
- (Até agora sem muita utilidade...)

- A tecnica chama Rainbow Table**s**


TXT
hash
TXT
hash
TXT
hash
TXT
TXT
hash
TXT
hash
TXT
hash
TXT
TXT
hash
TXT
hash
TXT
hash
TXT
TXT
hash
TXT
hash
TXT
hash
TXT
TXT
hash
TXT
hash
TXT
hash
TXT

- Basta adaptar o algoritmo para comparar com os varios ends
- Uma vez que a cadeia contendo h é encontrada o start é usado para encontrar o texto que gerou o h (Objetivo inicial!)
- Lembrando: tudo isso para economizar memoria em um ataque de pré-computação!! O custo total é maior que uma força bruta direta...

- O tradeoff é dado pelo “formato” da tabela
- Uma tabela "alta/estreita"gasta mais memoria...
- Uma tabela "baixa/larga"gasta mais CPU
  Com uma função de redução "perfeita", uma linha seria suficiente para inverter...mas com custo exponencial

- Alfabeto minúsculas, maiúsculas, números e caracteres especiais
  69 possibilidades
- Senha de 3 caracteres
  328509 possibilidades
- 8000 linhas com 600 de largura
  328509/(8000*600) = 0.068
- Maior que o espaço....
- apenas 3% da chaves foram encontradas

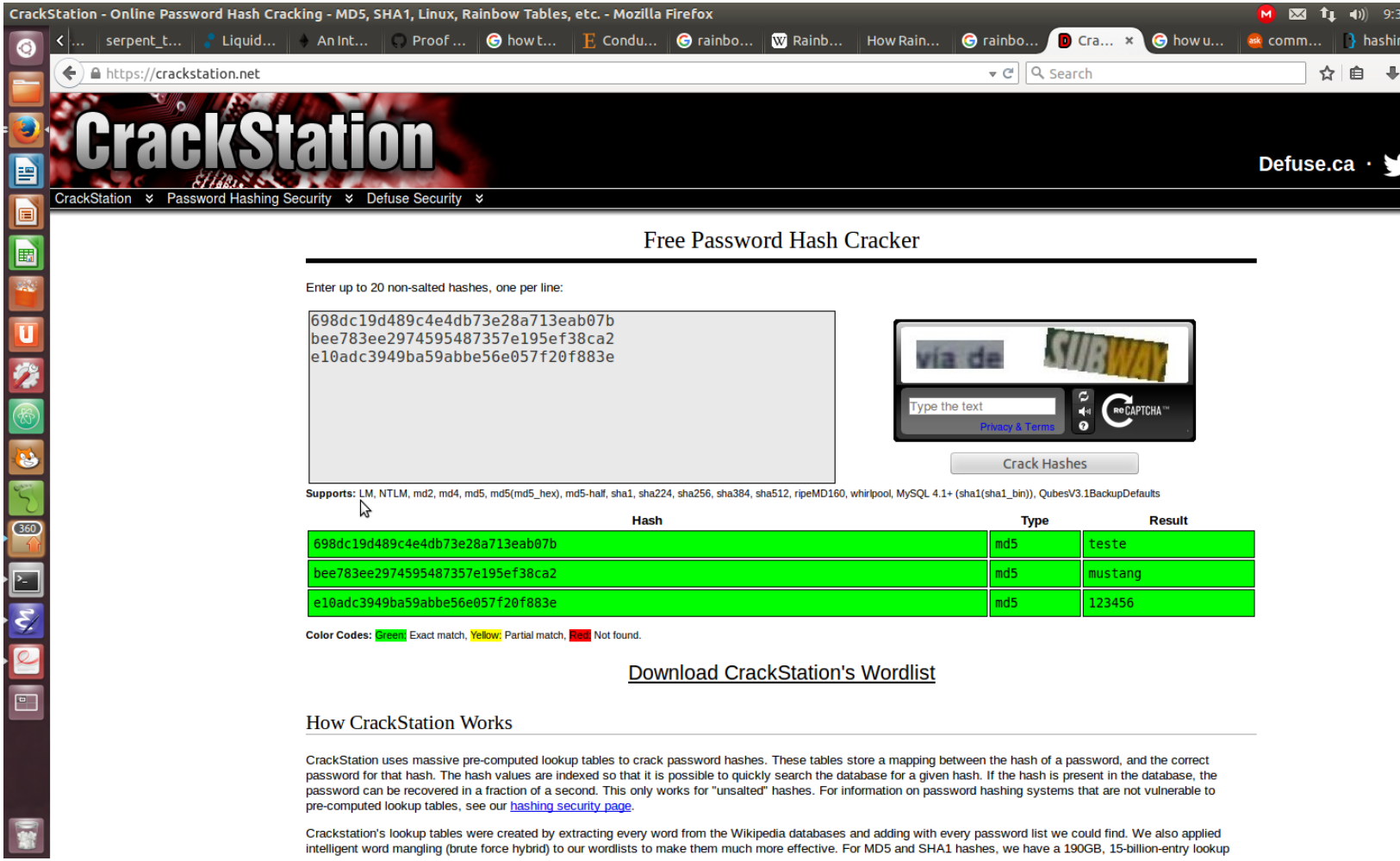
- Colisões
  Provável:
  $$\mathcal{R}(h_1) == \mathcal{R}(h_2)$$
- São mais frequentes do que gostaríamos
  Paradoxo do Aniversário!
- Não são **pontuais**
  Se colidir no "meio"da linha...vai até o fim
  Duas linhas (parcialmente) com a mesma informação

- A tecnica chama **Rainbow** Tables
- Cada coluna usa uma função de redução diferente
- Cada função é uma **cor**
- As colisões são pontuais

$$\mathcal{R}_i(h) \neq \mathcal{R}_j(h)$$

- http://project-rainbowcrack.com
- crackstation.net

```
luke@xwing:~$ echo -n teste | md5sum
698dc19d489c4e4db73e28a713eab07b  -
luke@xwing:~$ echo -n mustang | md5sum
bee783ee2974595487357e195ef38ca2  -
luke@xwing:~$ echo -n 123456 | md5sum
e10adc3949ba59abbe56e057f20f883e  -
luke@xwing:~$
```


CrackStation - Online Password Hash Cracking - MD5, SHA1, Linux, Rainbow Tables, etc. - Mozilla Firefox
https://crackstation.net
Search
CrackStation
Defuse.ca
CrackStation
Password Hashing Security
Defuse Security
Free Password Hash Cracker
Enter up to 20 non-salted hashes, one per line:
698dc19d489c4e4db73e28a713eab07b
bee783ee2974595487357e195ef38ca2
e10adc3949ba59abbe56e057f20f883e
Type the text
Privacy & Terms
Crack Hashes
Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults
Hash
Type
Result
698dc19d489c4e4db73e28a713eab07b
md5
teste
bee783ee2974595487357e195ef38ca2
md5
mustang
e10adc3949ba59abbe56e057f20f883e
md5
123456
Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.
Download CrackStation's Wordlist
How CrackStation Works
CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our hashing security page.
Crackstation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 190GB, 15-billion-entry lookup

Enter up to 20 non-salted hashes, one per line:

```
698dc19d489c4e4db73e28a713eab07b
bee783ee2974595487357e195ef38ca2
e10adc3949ba59abbe56e057f20f883e
```

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, w

Hash	Type	Result
698dc19d489c4e4db73e28a713eab07b	md5	teste
bee783ee2974595487357e195ef38ca2	md5	mustang
e10adc3949ba59abbe56e057f20f883e	md5	123456

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Resumo - Rainbow Tables

- Exemplo interessante de memória X tempo
- Caixa-Preta
- Tamanho do hash: não importa (dicionário)
- Velocidade do hash: SIM!

Resumo - Rainbow Tables

- Use salt
- Garanta que os seus usuários **não usem senhas de dicionário**
- (Política / Verificação)

Resumo - Rainbow Tables

- Key Derivation Function
- Argon2id

```
import os
from cryptography.hazmat.primitives.kdf.argon2 import Argon2id

salt = os.urandom(16)

kdf = Argon2id(
    salt=salt,
    length=32,
    iterations=100,
    lanes=4,
    memory_cost=64 * 1024)

password = b"segredo"
key = kdf.derive(password)

kdf_verify = Argon2id(
    salt=salt,
    length=32,
    iterations=100,
    lanes=4,
    memory_cost=64 * 1024)
kdf_verify.verify(password, key)
```

- Produzir uma RainbowTable(RT) capaz de quebrar senhas "pequenas"usando o Diceware
- Usar esta lista do Diceware (link)
- A função de redução deve produzir uma senha baseada no Diceware a partir do hash
- Varias funções diferentes (cores)
- O exercício consiste em explorar as possibilidades de uma RT
- Altura e "Largura"
- Testes aleatórios, eficácia, eficiência,...
- Descrição do Hardware é importante

```
def myHash(s):
    hash_object = hashlib.sha256()
    hash_object.update(b"GBC083")
    hash_object.update(s.encode('utf-8'))
    return hash_object.hexdigest()
    return  hash_object.digest()

#echo -n GBC0831234 | sha256sum
#5042f15f93d4f75df9cdff09d062d607eaa0140a930b9023c43ca7cc58609b8f
print(myHash("1234"))
```